Library provides `std::thread::hardware_concurrency()`. You've already seen how that can be used to scale the number of threads to the hardware.

Using `std::thread::hardware_concurrency()` directly requires care; your code doesn't take into account any of the other threads that are running on the system unless you explicitly share that information. In the worst case, if multiple threads call a function that uses `std::thread::hardware_concurrency()` for scaling at the same time, there will be huge oversubscription. `std::async()` avoids this problem because the library is aware of all calls and can schedule appropriately. Careful use of thread pools can also avoid this problem.

However, even if you take into account all threads running in your application, you're still subject to the impact of other applications running at the same time. Although the use of multiple CPU-intensive applications simultaneously is rare on single-user systems, there are some domains where it's more common. Systems designed to handle this scenario typically offer mechanisms to allow each application to choose an appropriate number of threads, although these mechanisms are outside the scope of the C++ Standard. One option is for a `std::async()`-like facility to take into account the total number of asynchronous tasks run by all applications when choosing the number of threads. Another is to limit the number of processing cores that can be used by a given application. I'd expect such a limit to be reflected in the value returned by `std::thread::hardware_concurrency()` on such platforms, although this isn't guaranteed. If you need to handle this scenario, consult your system documentation to see what options are available to you.

One additional twist to this situation is that the ideal algorithm for a problem can depend on the size of the problem compared to the number of processing units. If you have a *massively parallel* system with many processing units, an algorithm that performs more operations overall may finish more quickly than one that performs fewer operations, because each processor performs only a few operations.